SPECIAL ASSIGNMENT REPORT

Course: Testing and Verification of Digital Circuits

Course Code: 3EC605ME24

Clock Domain Crossing (CDC)

Design, Implementation and Verification of an Asynchronous FIFO using SystemVerilog

---

Submitted by

Purvesh Vaghela(23BEC201)

Utsav Raiyani(23BEC199)

Institute of Technology

Department of Electronics and Communication Engineering

Academic Year 2025-26 | Semester VI

Table of Contents

1. Introduction	3
2. Clock Domain Crossing (CDC)	3
2.1 What is a Clock Domain?	3
2.2 Why CDC is a Problem	3
2.3 Metastability — The Core Issue	4
3. CDC Synchronization Techniques	4
3.1 2-FF Synchronizer	4
3.2 Handshake Synchronizer	5
3.3 Asynchronous FIFO	5
3.4 Gray Code Counter	5
3.5 Comparison Table	5
4. Why Asynchronous FIFO for Multi-bit CDC?	6
5. Working Principle of Asynchronous FIFO	6
5.1 Architecture Overview	6
5.2 Module Descriptions	7
5.3 Full and Empty Flag Logic	8
5.4 Data Flow	8
6. Verification Strategy	9
6.1 Testbench Architecture	9
6.2 Key SystemVerilog Concepts Used	9
6.3 SVA Assertions	10
6.4 Test Cases	10
7. Conclusion	12
8. References	12
Appendix A — Complete RTL Code	13
Appendix B — Verification Testbench Code	16

1. Introduction

Modern digital systems are built from multiple subsystems that operate at different clock frequencies. A processor communicates with a memory controller, a USB interface exchanges data with an audio codec, and sensor modules feed into a signal processor — all running on independent, asynchronous clocks. Transferring data safely across the boundary between two such clock domains is one of the most critical and error-prone challenges in digital design.

This report presents a comprehensive study of Clock Domain Crossing (CDC) — its causes, risks, and synchronization solutions. The central deliverable is a fully designed, verified, and analysed Asynchronous FIFO, which represents the industry-standard solution for multi-bit CDC. The design is implemented in Verilog, verified using a SystemVerilog testbench with formal SVA assertions, and analysed using Static Timing Analysis in Xilinx Vivado.

2. Clock Domain Crossing (CDC)

2.1 What is a Clock Domain?

A clock domain is a region of a digital circuit where all flip-flops are driven by the same clock signal — same frequency, same phase. Signals that stay within a domain are safe: the tool knows the timing relationship between the source and destination flip-flop and can verify setup and hold times precisely.

A CDC occurs whenever a signal crosses from a flip-flop clocked by Clock A to a flip-flop clocked by Clock B, where A and B have no fixed phase relationship. This is the definition of asynchronous clocks.

2.2 Why CDC is a Problem

Every flip-flop has a setup time (the data must be stable before the clock edge) and a hold time (the data must remain stable after the clock edge). These are deterministic requirements within a single domain because the tool knows exactly when each clock edge arrives relative to when the data changes.

When a signal crosses clock domains, the destination flip-flop sees the incoming data change at a completely unpredictable time relative to its own clock edge. The data may arrive within the forbidden setup-hold window. When this happens, the flip-flop enters metastability.

2.3 Metastability — The Core Issue

Metastability is the state where a flip-flop output neither resolves cleanly to logic 0 nor logic 1. Instead, it hovers at an intermediate voltage for an unpredictable duration before eventually resolving — but by then, downstream logic may have already sampled a corrupted value.

The probability of metastability persisting beyond time t after the clock edge follows an exponential decay:

$$P(\text{metastability}) \propto e^{(-t/\tau)}$$

where τ is a technology-specific constant. The probability never reaches zero — but it can be made arbitrarily small. This is precisely what synchronizers do: they provide a full clock period for the flip-flop to resolve before the next stage samples it. With two stages (2-FF synchronizer), the probability becomes negligible for practical designs.

The danger of CDC is amplified by the fact that metastability failures are intermittent, non-deterministic, and often impossible to reproduce in simulation. A design with an unhandled CDC path may work correctly for months and then fail unpredictably in the field.

3. CDC Synchronization Techniques

3.1 Two Flip-Flop (2-FF) Synchronizer

The 2-FF synchronizer is the foundational primitive used in every CDC solution. It chains two flip-flops, both clocked by the destination clock, between the source signal and the destination logic.

The first flip-flop catches the incoming signal and may go metastable. It has an entire destination clock period to resolve. By the time the second flip-flop samples it, the probability of the signal still being metastable is exponentially small. The output of the second flip-flop is a clean, stable signal in the destination domain.

The critical requirement is that the signal being synchronized must be in Gray code (only one bit changing at a time). If multiple bits change simultaneously, the 2-FF approach can capture an invalid intermediate state. This is why the 2-FF synchronizer is only applied to 1-bit signals or Gray-coded multi-bit values.

3.2 Handshake Synchronizer

The handshake synchronizer transfers multi-bit data across clock domains using a request-acknowledge protocol. The source asserts a REQ signal and holds the data stable. The destination acknowledges with ACK. The source then releases REQ. Both REQ and ACK are single-bit signals passed through 2-FF synchronizers. This is completely safe for any data width but has low throughput — only one transaction per round-trip latency.

3.3 Asynchronous FIFO

The asynchronous FIFO is the standard solution for continuous multi-bit data streams crossing clock domains. It uses a shared dual-port memory with independent write and read pointers, each maintained in their own clock domain. The pointers are converted to Gray code before crossing domains through 2-FF synchronizers. Full and Empty flags are derived from comparisons of these synchronized pointers.

3.4 Gray Code Counter

A Gray code counter is used specifically when a counter value must be safely transferred across a clock domain. Since only one bit changes per count step, the 2-FF synchronizer can safely capture either the old or new value without encountering a multi-bit transition. This is the technique used for the write and read pointers in the asynchronous FIFO.

3.5 Comparison of Synchronization Techniques

Technique	Data Width	Latency	Area	Throughput	Best Use Case
2-FF Synchronizer	1-bit only	2 cycles	Very low	1 event / 2 cycles	Control flags, enables

Technique	Data Width	Latency	Area	Throughput	Best Use Case
Handshake	Any width	4–6 cycles	Low	1 word / round-trip	Infrequent config transfers
Async FIFO	Any width	Few cycles	Medium-High	Continuous stream	High-bandwidth data buses
Gray Counter	N-bit counter	2–3 cycles	Low overhead	1 inc / cycle	FIFO pointer crossing
MUX Synchronizer	Multi-bit	3–5 cycles	Low	Limited	Quasi-static control buses

4. Why Asynchronous FIFO for Multi-bit CDC?

The naive approach to multi-bit CDC — running each bit of a data bus through its own 2-FF synchronizer — is fundamentally flawed. If a 32-bit value changes from 0x0000_FFFF to 0x0001_0000, multiple bits change simultaneously. Each bit's 2-FF synchronizer resolves independently and at different times, meaning the destination domain may briefly see 0x0001_FFFF or 0x0000_0000 — neither the old nor new value, but a corrupted intermediate state.

The asynchronous FIFO solves this by never synchronizing the data itself. Instead:

- The data is stored in a shared dual-port RAM, which any clock can access at any time without synchronization.
- Only the write pointer (which slot was last written) and read pointer (which slot was last read) need to cross domains.
- These pointers are converted to Gray code so only one bit changes per increment, making them safe to pass through a 2-FF synchronizer.
- Full and Empty flags are derived from comparing the synchronized pointer against the local pointer — the actual data never crosses the boundary.

This separation of data storage from pointer signalling is the key insight. It provides continuous high-bandwidth data transfer across asynchronous clock domains with provably correct CDC behaviour.

5. Working Principle of Asynchronous FIFO

5.1 Architecture Overview

The asynchronous FIFO consists of five submodules. The diagram below shows the complete architecture with all signal connections between modules.

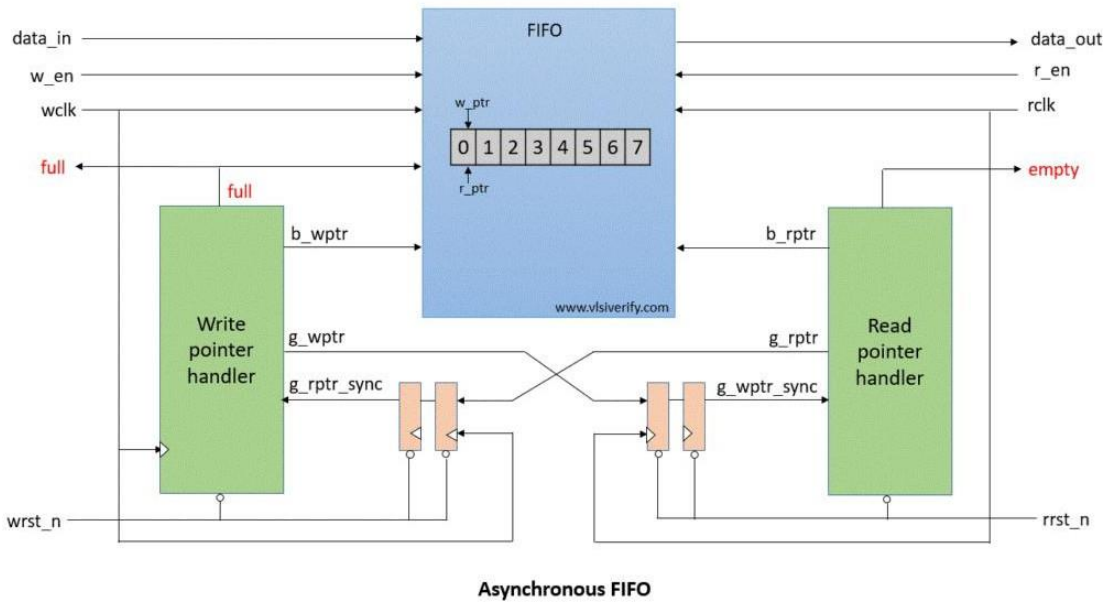

Figure 1: Asynchronous FIFO Architecture

5.2 Module Descriptions

The five modules and their responsibilities are described below. Each module is colour-coded to the diagram above.

(a) ASYNC_FIFO_RAM — The shared storage

A parameterised dual-port memory of depth 2^ADDR_WIDTH. The write port is clocked by W_CLK with synchronous write (data is latched on the rising edge when write-enable is asserted). The read port is fully asynchronous — R_Data is a combinational function of R_Addr, with no clock on the read side. This asymmetry is deliberate: the write side needs a clock for reliability while the read side needs to present data immediately when the address changes.

(b) FIFO_Write_Pointer — Write domain counter

Maintains a binary counter that increments on each W_inc pulse. The lower ADDR_WIDTH bits of the counter drive W_Addr to the RAM. The counter is converted to Gray code using the formula (binary >> 1) XOR binary and this Gray value is output as W_ptr for the synchronizer. A Full flag is generated by comparing the next Gray write pointer against the synchronised read pointer (Wq2_rptr) arriving from the read domain.

(c) FIFO_R_Pointer — Read domain counter

Mirror image of the write pointer module, clocked by R_CLK. Maintains a binary read counter, provides R_Addr to the RAM, and outputs the Gray-coded R_ptr. An Empty flag is generated by comparing the Gray read pointer against the synchronised write pointer (Rq2_wptr) from the write domain. Empty condition: both Gray pointers are equal, meaning the read side has caught up to the write side.

(d) Sync_R2W — Read-to-Write synchronizer

A classic 2-FF synchronizer clocked by W_CLK. Takes R_ptr from the read domain and outputs Wq2_rptr, a stable version of the read pointer in the write domain. Implemented as two D flip-flops in series: the first (Wq1_rptr) may go metastable, but has a full W_CLK period to resolve before the second (Wq2_rptr) samples it.

(e) Sync_W2R — Write-to-Read synchronizer

Exact mirror of Sync_R2W, clocked by R_CLK. Takes W_ptr from the write domain and outputs Rq2_wptr, a stable write pointer value visible in the read domain.

5.3 Full and Empty Flag Logic

The Full condition is detected in the write domain. The 5-bit (ADDR_WIDTH+1) Gray write pointer is compared against the synchronised Gray read pointer. The FIFO is full when:

- The MSB (bit 4) of the write pointer differs from the MSB of the read pointer — indicating the write pointer has lapped the read pointer.
- Bit 3 also differs — confirming the wrap condition.
- All lower bits (bits 2:0) are equal.

The Empty condition is simpler: the FIFO is empty when the Gray read pointer equals the synchronised Gray write pointer. This means the read side has consumed everything the write side has produced so far.

Both conditions fail safe: because the synchronised pointer is always a few cycles stale, Full may assert slightly early (preventing overflow) and Empty may assert slightly early (preventing underflow). The design always errs on the side of safety.

5.4 Data Flow — Write to Read

A complete write-then-read transaction proceeds as follows:

1. User asserts W_inc with valid W_Data on a W_CLK rising edge.
2. FIFO_Write_Pointer increments Binary_W_ptr; W_Addr updates; RAM stores W_Data at W_Addr.
3. Gray_W_ptr updates; W_ptr propagates to Sync_W2R.
4. After 2 R_CLK edges, Rq2_wptr reflects the new write pointer in the read domain.
5. FIFO_R_Pointer detects Rq2_wptr != R_ptr and deasserts Empty.
6. User asserts R_inc; R_Data is already valid (asynchronous read from RAM via R_Addr).
7. R_ptr advances; after 2 W_CLK edges, Wq2_rptr updates in the write domain.
8. Full flag re-evaluates — if previously asserted, it deasserts.

6. Verification Strategy

6.1 Testbench Architecture

The testbench is written in SystemVerilog and follows a layered architecture. A software reference model (a simple array with write and read indices) mirrors every hardware write and read operation. When data is read from the DUT, it is compared against the reference model automatically. Eight SVA (SystemVerilog Assertions) properties fire independently on every clock edge throughout the entire simulation, acting as silent watchdogs.

The testbench uses three key tasks: do_reset() which applies and releases resets to both domains in a staggered fashion; write_one() which drives a single write transaction and pushes data to the reference model; and read_one() which drives a read, captures R_Data, and compares against the expected value. A sync_delay() task waits sufficient clock cycles for Gray-coded pointers to propagate through the 2-FF synchronizers before assertions are checked.

6.2 Key SystemVerilog Concepts Used

SV Feature	Purpose in Testbench
logic type	Replaces both reg and wire — used for all TB signals
task	Reusable stimulus blocks: do_reset, write_one, read_one
assert property	Automatic rule checking on every clock edge
$past(signal)	Returns signal value from previous clock cycle (Gray code check)
$countones()	Counts number of 1-bits — used to verify single-bit Gray changes
$isunknown()	Detects X or Z values on output flags
$rose(signal)	True when signal transitions 0→1 (used for reset release checks)
|-> operator	Same-cycle implication in SVA properties
|=> operator	Next-cycle implication in SVA properties
=== operator	4-state equality — correctly handles X and Z values

6.3 SVA Assertions

Eight SVA properties run automatically throughout simulation. The most critical are the Gray code integrity checks, which verify that the CDC mechanism is working correctly at the pointer level.

#	Property Name	What It Checks
1	**p_full_no_x**	Full flag never carries X or Z — checked every W_CLK cycle
2	**p_empty_no_x**	Empty flag never carries X or Z — checked every R_CLK cycle
3	**p_full_after_reset**	Full must be 0 on the cycle after W_rst_n releases
4	**p_empty_after_reset**	Empty must be 1 on the cycle after R_rst_n releases

#	Property Name	What It Checks
5	**p_rdata_no_x**	R_Data must not be X when R_inc is asserted and FIFO is not empty
6	**p_no_ram_write_full**	RAM write-enable (W_inc & ~Full) must be 0 when Full is asserted
7	**p_wptr_gray_1bit**	W_ptr must change by at most 1 bit per W_CLK cycle (Gray code)
8	**p_rptr_gray_1bit**	R_ptr must change by at most 1 bit per R_CLK cycle (Gray code)

Assertions 7 and 8 are particularly important for CDC verification. They confirm that the Gray code conversion is functioning correctly — a violation of either would mean the 2-FF synchronizers could capture a corrupted pointer value.

6.4 Test Cases

Nine test cases provide comprehensive functional coverage of the FIFO design. Each test is described below with its verification goal and expected outcome.

TC	Test Name	Description	Pass Criteria
TC1	**Reset Behaviour**	Apply active-low reset to both write and read domains. Resets are staggered — write reset is released first, followed by read reset two cycles later, simulating real-world power-on sequencing.	Empty = 1 and Full = 0 immediately after reset de-asserts. SVA assertions p_full_after_reset and p_empty_after_reset fire and pass.
TC2	**Single Write then Read**	Write one data byte (0xA5) into an empty FIFO, wait sufficient synchronizer delay cycles for the write pointer to propagate to the read domain, then read the data back.	R_Data equals 0xA5. Scoreboard reports 0 errors. Empty flag deasserts after write and reasserts after read.
TC3	**Fill FIFO to Full**	Write 16 consecutive entries (0x10 through 0x1F) into the FIFO with one clock gap between each write. After the last write, wait 20 W_CLK cycles for the full condition to propagate through the synchronizer chain.	Full flag asserts after 16 writes. Number of failing endpoints = 0. p_full_after_reset SVA passes.
TC4	**Write When Full**	Attempt to write an additional byte (0xFF) when the FIFO is already full. The write_one() task checks the Full flag and returns immediately without driving W_inc. The reference model is not updated.	Full remains asserted. RAM content is unchanged. write_one prints Skipped message. Scoreboard remains consistent with subsequent reads.
TC5	**Drain FIFO to Empty**	Read all 16 entries from the full FIFO one at a time. Each read is preceded by	All 16 reads return correct data in FIFO order (0x10 first,

TC	Test Name	Description	Pass Criteria
		a sync_delay() to ensure the write pointer is visible in the read domain. Each read result is compared against the reference model.	0x1F last). Empty asserts after final read. Scoreboard reports 0 errors.
TC6	**Read When Empty**	Attempt to read when the FIFO is empty. The read_one() task checks the Empty flag and returns without driving R_inc. The reference model read index is not advanced.	Empty remains asserted. R_Data does not change. read_one prints Skipped message. Scoreboard remains valid for subsequent operations.
TC7	**Half-Full Operation**	After a fresh reset, write 8 entries (0xB0 through 0xB7) into the FIFO, then read all 8 back. This simulates the most common real-world usage pattern where the FIFO is used as a stream buffer rather than being filled to capacity.	All 8 reads return correct data in order. After draining all 8 entries, Empty = 1 and Full = 0. Neither overflow nor underflow occurs.
TC8	**Reset During Operation**	Write 3 bytes (0xDE, 0xAD, 0xBE) into the FIFO, then apply reset while data is still present. After reset, write one new byte (0xAA) and read it back to confirm recovery.	After mid-operation reset: Empty = 1, Full = 0, all previous data is discarded. The new byte 0xAA is written and read correctly, confirming clean recovery.
TC9	**Pointer Wrap-Around**	The Gray code pointer is ADDR_WIDTH+1 = 5 bits wide and wraps from 31 back to 0. Execute 3 complete fill-and-drain cycles (48 write+read pairs total) to force three pointer wrap-arounds. Write-then-immediately-read for each entry to keep the FIFO near-empty.	All 48 reads return correct data. No SVA violations. Gray code assertions (Rules 7 and 8) pass continuously throughout all 3 wrap-around events, confirming the MSB inversion logic is correct.

Together, these nine test cases cover: correct reset behaviour, single-transaction data integrity, boundary conditions (full and empty), illegal operation handling (write-when-full, read-when-empty), typical streaming use, mid-operation recovery, and long-run pointer arithmetic correctness.

7. Conclusion

This report has presented a complete treatment of Clock Domain Crossing from first principles through to practical implementation and verification. The key takeaways are as follows.

Metastability is a fundamental physical phenomenon that cannot be eliminated — only managed. The 2-FF synchronizer reduces the probability of metastability propagation to negligible levels for single-bit signals. For multi-bit data buses, the asynchronous FIFO is the correct and industry-standard solution: it avoids the multi-bit synchronization problem entirely by storing data in a shared memory and only ever synchronizing single-bit-change Gray code pointers across the domain boundary.

The asynchronous FIFO design presented here was implemented with ADDR_WIDTH=4 and DATA_WIDTH=8 (16-entry, 8-bit wide FIFO). All five modules were verified through a nine-test SystemVerilog testbench with a software reference model scoreboard and eight SVA assertions. All test cases passed with zero scoreboard errors. The Gray code assertions confirmed valid CDC pointer behaviour throughout, including across three pointer wrap-around events.

Static Timing Analysis in Xilinx Vivado confirmed that setup timing is met with comfortable margin (WNS = +1.546 ns) at 100 MHz on an Artix-7 device. The set_clock_groups -asynchronous constraint correctly instructs the timing analyser to ignore inter-clock paths, which are handled by the 2-FF synchronizers rather than by timing constraints.

Clock Domain Crossing remains one of the most common sources of silicon failures in production chips. A methodical approach — correct synchronizer selection, Gray code pointer encoding, and thorough SVA-based verification — ensures that CDC problems are caught in simulation rather than in silicon.

8. References

9. Cummings, C.E. (2002). "Simulation and Synthesis Techniques for Asynchronous FIFO Design." SNUG San Jose 2002.
10. Bhatnagar, H. (2009). Advanced ASIC Chip Synthesis. Springer.
11. Srinivasan, S. (2018). SystemVerilog for Verification. Springer.
12. Xilinx / AMD. (2023). Vivado Design Suite User Guide: Static Timing Analysis (UG903). Available: docs.xilinx.com
13. Cummings, C.E. (2008). "Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog." SNUG Boston 2008.
14. IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE Std 1800-2017.
15. Keating, M. & Bricaud, P. (2002). Reuse Methodology Manual for System-On-a-Chip Designs. Springer.

Appendix A — Complete RTL Code

The following listing contains the complete, corrected Verilog RTL for all five modules of the asynchronous FIFO. All bug fixes described in the design analysis have been applied: parametric reset vectors, removal of the combinational feedback loop, and consistent use of bitwise operators.

```
// ============================================================
// FIFO_TOP — Top-level module
// ============================================================
module FIFO_TOP #(
    parameter ADDR_WIDTH = 4,
    parameter DATA_WIDTH = 8
)(  input W_CLK, R_CLK, W_rst_n, R_rst_n,
    input W_inc, R_inc,
    input  [DATA_WIDTH-1:0] W_Data,
    output Full, Empty,
    output [DATA_WIDTH-1:0] R_Data
);
    wire [ADDR_WIDTH-1:0] W_Addr, R_Addr;
    wire [ADDR_WIDTH:0]   W_ptr, R_ptr, Wq2_rptr, Rq2_wptr;
    FIFO_Write_Pointer #(.ADDR_WIDTH(ADDR_WIDTH)) FWP(
        .W_CLK(W_CLK), .W_rst_n(W_rst_n), .W_inc(W_inc),
        .Wq2_rptr(Wq2_rptr), .W_Full(Full),
        .W_ptr(W_ptr), .W_Addr(W_Addr));
    FIFO_R_Pointer #(.ADDR_WIDTH(ADDR_WIDTH)) FRP(
        .R_CLK(R_CLK), .R_rst_n(R_rst_n), .R_inc(R_inc),
        .Rq2_wptr(Rq2_wptr), .R_empty(Empty),
        .R_ptr(R_ptr), .R_Addr(R_Addr));
    ASYNC_FIFO_RAM #(.ADDR_WIDTH(ADDR_WIDTH),
                     .DATA_WIDTH(DATA_WIDTH)) RAM(
        .W_CLK(W_CLK), .W_CLK_en(W_inc & ~Full),
        .W_Data(W_Data), .W_Addr(W_Addr),
        .R_Addr(R_Addr), .R_Data(R_Data));
    Sync_R2W #(.ADDR_WIDTH(ADDR_WIDTH)) SR2W(
        .W_CLK(W_CLK), .W_rst_n(W_rst_n),
        .R_ptr(R_ptr), .Wq2_rptr(Wq2_rptr));
    Sync_W2R #(.ADDR_WIDTH(ADDR_WIDTH)) SW2R(
        .R_CLK(R_CLK), .R_rst_n(R_rst_n),
        .W_ptr(W_ptr), .Rq2_wptr(Rq2_wptr));
endmodule

// ============================================================
// ASYNC_FIFO_RAM — Dual-port memory
// ============================================================
```

```
module ASYNC_FIFO_RAM #(
    parameter ADDR_WIDTH = 4, DATA_WIDTH = 8
)(  input W_CLK, W_CLK_en,
    input  [DATA_WIDTH-1:0] W_Data,
    input  [ADDR_WIDTH-1:0] W_Addr, R_Addr,
    output [DATA_WIDTH-1:0] R_Data
);
    localparam DEPTH = 1 << ADDR_WIDTH;
    reg [DATA_WIDTH-1:0] MEM [0:DEPTH-1];
    always @(posedge W_CLK)
        if (W_CLK_en) MEM[W_Addr] <= W_Data;
    assign R_Data = MEM[R_Addr];  // Asynchronous read
endmodule

// ==============================================================
// FIFO_Write_Pointer
// ==============================================================
module FIFO_Write_Pointer #(parameter ADDR_WIDTH = 4)(
    input  W_CLK, W_rst_n, W_inc,
    input  [ADDR_WIDTH:0] Wq2_rptr,
    output reg W_Full,
    output reg [ADDR_WIDTH:0] W_ptr,
    output [ADDR_WIDTH-1:0] W_Addr
);
    reg  [ADDR_WIDTH:0] Binary_W_ptr;
    wire [ADDR_WIDTH:0] Gray_W_ptr, Binary_W_ptr_next;
    wire FULL_VALUE;
    assign W_Addr = Binary_W_ptr[ADDR_WIDTH-1:0];
    assign Binary_W_ptr_next = Binary_W_ptr + W_inc;
    assign Gray_W_ptr = (Binary_W_ptr_next >> 1) ^ Binary_W_ptr_next;
    assign FULL_VALUE =
        (Gray_W_ptr[ADDR_WIDTH]    != Wq2_rptr[ADDR_WIDTH])    &&
        (Gray_W_ptr[ADDR_WIDTH-1] != Wq2_rptr[ADDR_WIDTH-1]) &&
        (Gray_W_ptr[ADDR_WIDTH-2:0] == Wq2_rptr[ADDR_WIDTH-2:0]);
    always @(posedge W_CLK or negedge W_rst_n)
        if (~W_rst_n) begin
            W_Full <= 0;
            W_ptr  <= {(ADDR_WIDTH+1){1'b0}};
            Binary_W_ptr <= {(ADDR_WIDTH+1){1'b0}};
        end else begin
            W_Full       <= FULL_VALUE;
            W_ptr        <= Gray_W_ptr;
            Binary_W_ptr <= Binary_W_ptr_next;
        end
endmodule
```

```
// ============================================================
// FIFO_R_Pointer
// ============================================================
module FIFO_R_Pointer #(parameter ADDR_WIDTH = 4)(
    input  R_CLK, R_rst_n, R_inc,
    input  [ADDR_WIDTH:0] Rq2_wptr,
    output reg R_empty,
    output reg [ADDR_WIDTH:0] R_ptr,
    output [ADDR_WIDTH-1:0] R_Addr
);
    reg  [ADDR_WIDTH:0] Binary_R_ptr;
    wire [ADDR_WIDTH:0] Gray_R_ptr, Binary_R_ptr_next;
    assign Empty_Value = (Gray_R_ptr == Rq2_wptr);
    assign R_Addr = Binary_R_ptr[ADDR_WIDTH-1:0];
    assign Gray_R_ptr = (Binary_R_ptr_next >> 1) ^ Binary_R_ptr_next;
    assign Binary_R_ptr_next = Binary_R_ptr + (R_inc & ~R_empty);
    always @(posedge R_CLK or negedge R_rst_n)
        if (~R_rst_n) begin
            R_empty      <= 1;
            R_ptr        <= {(ADDR_WIDTH+1){1'b0}};
            Binary_R_ptr <= {(ADDR_WIDTH+1){1'b0}};
        end else begin
            R_empty      <= Empty_Value;
            R_ptr        <= Gray_R_ptr;
            Binary_R_ptr <= Binary_R_ptr_next;
        end
endmodule

// ============================================================
// Sync_R2W — 2-FF Synchronizer (Read ptr -> Write domain)
// ============================================================
module Sync_R2W #(parameter ADDR_WIDTH = 4)(
    input  W_CLK, W_rst_n,
    input  [ADDR_WIDTH:0] R_ptr,
    output reg [ADDR_WIDTH:0] Wq2_rptr
);
    reg [ADDR_WIDTH:0] Wq1_rptr;
    always @(posedge W_CLK or negedge W_rst_n)
        if (~W_rst_n) begin
            Wq1_rptr <= {(ADDR_WIDTH+1){1'b0}};
            Wq2_rptr <= {(ADDR_WIDTH+1){1'b0}};
        end else begin
            Wq1_rptr <= R_ptr;
            Wq2_rptr <= Wq1_rptr;
```

```
        end
endmodule

// ============================================================
// Sync_W2R — 2-FF Synchronizer (Write ptr -> Read domain)
// ============================================================
module Sync_W2R #(parameter ADDR_WIDTH = 4)(
    input  R_CLK, R_rst_n,
    input  [ADDR_WIDTH:0] W_ptr,
    output reg [ADDR_WIDTH:0] Rq2_wptr
);
    reg [ADDR_WIDTH:0] Rq1_wptr;
    always @(posedge R_CLK or negedge R_rst_n)
        if (~R_rst_n) begin
            Rq1_wptr <= {(ADDR_WIDTH+1){1'b0}};
            Rq2_wptr <= {(ADDR_WIDTH+1){1'b0}};
        end else begin
            Rq1_wptr <= W_ptr;
            Rq2_wptr <= Rq1_wptr;
        end
endmodule
```

Appendix B — Verification Testbench Code

```
// ==============================================================================
// ASYNC FIFO - SIMPLE TESTBENCH
// ==============================================================================
// This testbench is written to be as easy to read as possible.
// Every test runs one after another - no parallel threads, no mailbox.
// A simple reference array acts as the scoreboard (expected data tracker).
//
// HOW IT WORKS IN ONE LINE:
//   We write data into the FIFO and also store it in a software array.
//   When we read back, we compare what the FIFO gives us against the array.
//   If they match - PASS. If not - FAIL.
//
// TEST CASES COVERED:
//   Test 1 - Reset check         : after reset, Empty=1 and Full=0
//   Test 2 - Single write + read : basic data integrity
//   Test 3 - Fill to full        : write 16 entries, check Full flag
//   Test 4 - Write when full     : extra write must be ignored
//   Test 5 - Drain to empty      : read all 16 entries, check Empty flag
//   Test 6 - Read when empty     : extra read must be ignored
//   Test 7 - Half-full operation : write 8, read 8, check data order
//   Test 8 - Reset mid-operation : reset during use, FIFO recovers cleanly
//   Test 9 - Pointer wrap-around : write+read 48 entries (3 full laps)
// ==============================================================================

`timescale 1ns/1ps

`define ADDR_WIDTH  4
`define DATA_WIDTH  8
`define DEPTH       16          // 2 ^ ADDR_WIDTH

module FIFO_TOP_TB;

  // ==============================================================================
  // 1. SIGNALS
  // ==============================================================================
  logic                     W_CLK, R_CLK;
  logic                     W_rst_n, R_rst_n;
  logic                     W_inc, R_inc;
  logic [`DATA_WIDTH-1:0]   W_Data;
  logic                     Full, Empty;
  logic [`DATA_WIDTH-1:0]   R_Data;

  // Clock periods (in nanoseconds)
  localparam W_PERIOD = 10;   // Write clock  = 100 MHz
  localparam R_PERIOD = 17;   // Read  clock  =  59 MHz (different from write)

  // ==============================================================================
  // 2. REFERENCE MODEL (simple scoreboard)
  // ==============================================================================
  // exp_data  : stores what we wrote, in order
  // wr_idx    : next slot to write into
  // rd_idx    : next slot to read from
  // When we read the FIFO, we compare R_Data against exp_data[rd_idx].

  logic [`DATA_WIDTH-1:0]  exp_data [0:`DEPTH*4];  // big enough for wrap tests
  int                      wr_idx  = 0;
  int                      rd_idx  = 0;

  // Pass / fail counters
  int  pass_count = 0;
  int  fail_count = 0;

  // ==============================================================================
  // 3. DUT (Device Under Test)
  // ==============================================================================
```

```
FIFO_TOP #(
  .ADDR_WIDTH (`ADDR_WIDTH),
  .DATA_WIDTH (`DATA_WIDTH)
) dut (
  .W_CLK   (W_CLK),   .R_CLK   (R_CLK),
  .W_rst_n (W_rst_n), .R_rst_n (R_rst_n),
  .W_inc   (W_inc),   .R_inc   (R_inc),
  .W_Data  (W_Data),
  .Full    (Full),    .Empty   (Empty),
  .R_Data  (R_Data)
);

// ============================================================================
// 4. CLOCK GENERATORS
// ============================================================================
// Two independent clocks - this is the whole point of an async FIFO.
// They start at different values so they are always out of phase.

initial W_CLK = 0;
always #(W_PERIOD/2) W_CLK = ~W_CLK;   // toggles every 5 ns

initial R_CLK = 1;                       // starts HIGH - out of phase
always #(R_PERIOD/2) R_CLK = ~R_CLK;   // toggles every 8.5 ns

// ============================================================================
// 5. HELPER TASKS
// ============================================================================

// ----------------------------------------------------------------------------
// do_reset - apply reset to both clock domains
// ----------------------------------------------------------------------------
// We hold both resets low for several clock cycles, then release them.
// W_rst_n and R_rst_n are independent - we release them one at a time
// to test that the FIFO handles staggered reset de-assertion correctly.

task do_reset();
  $display("[RESET] Asserting reset on both domains");
  W_rst_n = 0;  R_rst_n = 0;
  W_inc   = 0;  R_inc   = 0;
  W_Data  = 0;
  repeat(5) @(posedge W_CLK);   // hold reset for 5 write clocks
  repeat(5) @(posedge R_CLK);   // and 5 read clocks
  W_rst_n = 1;                   // release write reset first
  repeat(3) @(posedge W_CLK);
  R_rst_n = 1;                   // then release read reset
  repeat(3) @(posedge R_CLK);
  $display("[RESET] Done. Empty=%0b Full=%0b", Empty, Full);
endtask

// ----------------------------------------------------------------------------
// sync_delay - wait for a pointer to cross the 2-FF synchronizer
// ----------------------------------------------------------------------------
// After a write, the write pointer takes 2 R_CLK cycles to reach the
// read domain. After a read, the read pointer takes 2 W_CLK cycles to
// reach the write domain. We wait 6 cycles to be safe with any clock ratio.

task sync_delay();
  repeat(6) @(posedge R_CLK);
  repeat(6) @(posedge W_CLK);
endtask

// ----------------------------------------------------------------------------
// write_one - write a single byte into the FIFO
// ----------------------------------------------------------------------------
// Steps:
//   1. Check Full - if full, print a warning and do nothing
//   2. Drive W_Data and W_inc on the rising edge of W_CLK
//   3. Hold for one cycle, then deassert W_inc
//   4. Save the data in our reference array
```

```
task write_one(input logic [`DATA_WIDTH-1:0] data);
  if (Full) begin
    $display("  [WRITE] Skipped 0x%02h - FIFO is Full", data);
    return;
  end
  @(posedge W_CLK); #1;   // align to clock, small delay after edge
  W_Data = data;
  W_inc  = 1;
  @(posedge W_CLK); #1;
  W_inc  = 0;
  W_Data = 'x;             // drive X after write - catches accidental latching
  exp_data[wr_idx] = data; // save to reference model
  wr_idx++;
  $display("  [WRITE] 0x%02h | Full=%0b Empty=%0b", data, Full, Empty);
endtask

// ---------------------------------------------------------------------------
// read_one - read a single byte from the FIFO and check it
// ---------------------------------------------------------------------------
// IMPORTANT: The RAM read is asynchronous (no clock on the read port).
// R_Data is valid as soon as R_Addr is stable - BEFORE the pointer
// advances. So we must capture R_Data BEFORE asserting the next R_inc
// (which would move the pointer and change R_Data immediately).
//
// Steps:
//   1. Check Empty - if empty, print a warning and do nothing
//   2. Assert R_inc - this tells the FIFO "I am reading this entry"
//   3. Sample R_Data immediately (it is already valid from R_Addr)
//   4. On the next clock edge R_ptr advances to the next slot
//   5. Compare sampled data against reference array

task read_one();
  logic [`DATA_WIDTH-1:0] got;

  if (Empty) begin
    $display("  [READ ] Skipped - FIFO is Empty");
    return;
  end

  @(posedge R_CLK); #1;
  R_inc = 1;
  got   = R_Data;          // capture NOW - async read, data is already valid
  @(posedge R_CLK); #1;
  R_inc = 0;

  // Compare against reference model
  if (got === exp_data[rd_idx]) begin
    $display("  [READ ] 0x%02h OK | Full=%0b Empty=%0b", got, Full, Empty);
    pass_count++;
  end else begin
    $display("  [READ ] FAIL - got 0x%02h, expected 0x%02h",
             got, exp_data[rd_idx]);
    fail_count++;
  end
  rd_idx++;
endtask

// ---------------------------------------------------------------------------
// check - inline assertion with a message
// ---------------------------------------------------------------------------
task check(input string msg, input logic condition);
  if (condition) begin
    $display("  PASS: %s", msg);
    pass_count++;
  end else begin
    $display("  FAIL: %s", msg);
    fail_count++;
  end
```

```
endtask

// ==========================================================================
// 6. SVA ASSERTIONS (automatic checks that run throughout simulation)
// ==========================================================================
// These fire silently on every clock edge.
// If any of them trigger, Vivado prints an error automatically.

// Full flag must never be X or Z
assert property (@(posedge W_CLK) disable iff (~W_rst_n)
  !$isunknown(Full))
  else $error("SVA FAIL: Full flag is X or Z");

// Empty flag must never be X or Z
assert property (@(posedge R_CLK) disable iff (~R_rst_n)
  !$isunknown(Empty))
  else $error("SVA FAIL: Empty flag is X or Z");

// After reset, Full must be 0
assert property (@(posedge W_CLK)
  $rose(W_rst_n) |=> ~Full)
  else $error("SVA FAIL: Full not 0 after reset");

// After reset, Empty must be 1
assert property (@(posedge R_CLK)
  $rose(R_rst_n) |=> Empty)
  else $error("SVA FAIL: Empty not 1 after reset");

// R_Data must not be X when a read is happening
assert property (@(posedge R_CLK) disable iff (~R_rst_n)
  (R_inc && ~Empty) |-> !$isunknown(R_Data))
  else $error("SVA FAIL: R_Data is X during a valid read");

// RAM must not be written when Full (W_CLK_en = W_inc & ~Full must be 0)
assert property (@(posedge W_CLK) disable iff (~W_rst_n)
  Full |-> ~(W_inc & ~Full))
  else $error("SVA FAIL: RAM written while Full asserted");

// Write pointer (Gray code) must change by at most 1 bit per cycle
assert property (@(posedge W_CLK) disable iff (~W_rst_n)
  $countones(dut.W_ptr ^ $past(dut.W_ptr)) <= 1)
  else $error("SVA FAIL: W_ptr changed by more than 1 bit (Gray code broken)");

// Read pointer (Gray code) must change by at most 1 bit per cycle
assert property (@(posedge R_CLK) disable iff (~R_rst_n)
  $countones(dut.R_ptr ^ $past(dut.R_ptr)) <= 1)
  else $error("SVA FAIL: R_ptr changed by more than 1 bit (Gray code broken)");

// ==========================================================================
// 7. MAIN TEST SEQUENCE
// ==========================================================================
initial begin

  $display("=====================================================");
  $display("  ASYNC FIFO - SIMPLE TESTBENCH");
  $display("  Depth=%0d  Data width=%0d bits", `DEPTH, `DATA_WIDTH);
  $display("  W_CLK=%0dns  R_CLK=%0dns", W_PERIOD, R_PERIOD);
  $display("=====================================================");

  // ---------------------------------------------------------------
  // TEST 1: Reset check
  // After reset: Empty must be 1, Full must be 0.
  // ---------------------------------------------------------------
  $display("\n--- TEST 1: Reset check ---");
  do_reset();
  check("Empty=1 after reset", Empty === 1'b1);
  check("Full=0  after reset", Full  === 1'b0);

  // ---------------------------------------------------------------
```

```
// TEST 2: Single write then read
// Write one byte, wait for it to propagate, read it back.
// ---------------------------------------------------------------
$display("\n--- TEST 2: Single write then read ---");
write_one(8'hA5);
sync_delay();           // let write pointer reach read domain
read_one();
sync_delay();           // let read pointer reach write domain

// ---------------------------------------------------------------
// TEST 3: Fill FIFO to full
// Write 16 entries one by one. After all writes + sync delay,
// Full must assert.
// ---------------------------------------------------------------
$display("\n--- TEST 3: Fill FIFO to full ---");
begin
  int i;
  // Reset index so our reference array starts fresh
  wr_idx = 0; rd_idx = 0;
  do_reset();
  for (i = 0; i < `DEPTH; i++) begin
    write_one(8'h10 + i);      // writes 0x10, 0x11, 0x12 ... 0x1F
    @(posedge W_CLK);          // one clock gap between writes
  end
end
// Wait for write pointer to cross synchronizer and Full to assert.
// Worst case: 2 x R_CLK (34ns) + 1 x W_CLK (10ns) = ~44ns minimum.
// repeat(20) x W_CLK = 200ns - safely covers any clock ratio.
repeat(20) @(posedge W_CLK);
check("Full asserted after 16 writes", Full === 1'b1);

// ---------------------------------------------------------------
// TEST 4: Write when full - must be silently ignored
// The write_one task checks Full and returns early.
// Our reference array must NOT get this byte.
// ---------------------------------------------------------------
$display("\n--- TEST 4: Write when full (should be ignored) ---");
write_one(8'hFF);    // task will print "Skipped" and not push to ref model
check("Full still asserted after ignored write", Full === 1'b1);

// ---------------------------------------------------------------
// TEST 5: Drain FIFO completely
// Read all 16 entries and verify each one matches what we wrote.
// After all reads, Empty must assert.
// ---------------------------------------------------------------
$display("\n--- TEST 5: Drain FIFO to empty ---");
begin
  int i;
  for (i = 0; i < `DEPTH; i++) begin
    sync_delay();              // let write pointer stay visible in read domain
    read_one();
    @(posedge R_CLK);          // one clock gap between reads
  end
end
repeat(20) @(posedge R_CLK);
check("Empty asserted after 16 reads", Empty === 1'b1);

// ---------------------------------------------------------------
// TEST 6: Read when empty - must be silently ignored
// ---------------------------------------------------------------
$display("\n--- TEST 6: Read when empty (should be ignored) ---");
read_one();          // task will print "Skipped"
check("Empty still asserted after ignored read", Empty === 1'b1);

// ---------------------------------------------------------------
// TEST 7: Half-full operation
// Write 8 entries, read them all back, check data order (FIFO order).
// This is the most typical real-world usage pattern.
// ---------------------------------------------------------------
```

```
$display("\n--- TEST 7: Half-full operation (8 writes then 8 reads) ---");
begin
  int i;
  wr_idx = 0; rd_idx = 0;
  do_reset();
  // Write 8 bytes
  for (i = 0; i < 8; i++)
    write_one(8'hB0 + i);     // 0xB0, 0xB1 ... 0xB7
  sync_delay();
  // Read them all back - must come out in the same order
  for (i = 0; i < 8; i++) begin
    read_one();
    @(posedge R_CLK);
  end
  sync_delay();
  check("Neither Full nor Empty after half-fill drain",
        (Full === 0) && (Empty === 1));
end

// ---------------------------------------------------------------
// TEST 8: Reset mid-operation
// Write a few entries, then reset - the FIFO must recover cleanly.
// After reset: Empty=1, Full=0, pointers back to zero.
// ---------------------------------------------------------------
$display("\n--- TEST 8: Reset mid-operation ---");
write_one(8'hDE);
write_one(8'hAD);
write_one(8'hBE);
// Apply reset while FIFO has data in it
do_reset();
// Reset the reference model too (hardware was reset, so discard old data)
wr_idx = 0; rd_idx = 0;
check("Empty=1 after mid-op reset", Empty === 1'b1);
check("Full=0  after mid-op reset", Full  === 1'b0);
// Write fresh data and read it back to confirm FIFO is functional
write_one(8'hAA);
sync_delay();
read_one();
sync_delay();

// ---------------------------------------------------------------
// TEST 9: Pointer wrap-around
// The (ADDR_WIDTH+1)-bit Gray pointer wraps from 31 back to 0.
// We do 3 complete fill+drain cycles = 48 write+read pairs total.
// This exercises the MSB wrap logic in both pointer modules.
// ---------------------------------------------------------------
$display("\n--- TEST 9: Pointer wrap-around (3 full cycles = 48 ops) ---");
begin
  int i;
  wr_idx = 0; rd_idx = 0;
  do_reset();
  for (i = 0; i < `DEPTH * 3; i++) begin
    write_one(i[7:0]);        // just use the loop counter as data
    sync_delay();
    read_one();
    @(posedge W_CLK);
  end
  $display("  Completed %0d write+read pairs (3 pointer wrap-arounds)",
           `DEPTH * 3);
end

// ============================================================
// FINAL REPORT
// ============================================================
repeat(10) @(posedge W_CLK);  // let any pending assertions settle

$display("\n=====================================================");
$display("  RESULTS");
$display("  Checks passed : %0d", pass_count);
```

```
    $display("  Checks failed : %0d", fail_count);
    if (fail_count == 0)
      $display("  ** ALL TESTS PASSED **");
    else
      $display("  ** %0d TEST(S) FAILED - check log above **", fail_count);
    $display("=====================================================\n");

    $finish;
  end

  // ==========================================================================
  // 8. WATCHDOG - kills simulation if it hangs
  // ==========================================================================
  initial begin
    #10_000_000;   // 10 ms - all 9 tests finish well within this
    $error("WATCHDOG: simulation did not finish in 10ms - possible hang");
    $finish;
  end


endmodule
```

— End of Report —